Citect Pty Ltd ABN 88 001 158 854
3 Fitzsimons Lane Gordon 2072 NSW Australia
PO Box 174 Pymble 2073 NSW Australia
Tel 61 2 9496 7300 Fax 61 2 9496 7399 www.citect.com

CitectSCADA

v5.4x

MOSCAD driver, User Information and Design

1 Driver Spec Version History

Author	Date	Comment
Stephen Burman	21/10/97	
Stephen Burman	24/2/98	
Stephen Burman	21/3/98	
David Rossini	18/05/98	
David Rossini	21/09/98	Enhancements for release of V2.01b1
David Rossini	22/09/98	Technical Enhancements for release of V2.01b1
David Rossini	14/04/99	Enhancements for release of V2.01b2
Lin Gu	10/11/00	Modified for V3.00.00.001 B1
JaeWoo Kim	29/11/01	Modified for V3.01.03.002 B1 (Refer to the Appendix)
Ben Liquete	08/03/02	Updated User Interface to include info re: Gateway Redundancy
Paul Nguyen/Lin Gu	2/09/2002	Gateway redundancy
Greg Roberts	22/10/02	Updated to include Gentronics changes in Holland by David Rossini, document to be reviewed by David when he returns. Information valid for version 3.2.x.x

1.1 Contents

1 DRIVER SPEC VERSION HISTORY 2
1.1 Contents 3
2 TARGET DEVICE(S) AND PROTOCOL 5
2.1 Introduction 5
2.2 Device Manufacturer 5
2.3 Device Definition 5
2.4 Communications Method 5
2.5 Communications/Hardware Configuration 6
2.5.1 Wiring Diagrams 6
2.5.2 I/O Device Settings 6
2.5.3 Software Setup 6
There is a MOSCAD Programming Toolbox available to configure gateways and another to configure RTUs. The computer running the Toolbox uses an RS-232 connection, and can be connected to either a gateway or an RTU. 6
2.5.3.1 Gateway Software Configuration 6
2.5.3.2 Gateway Mode of Operation Configuration 7
2.5.3.3 RTU Software Configuration 7
2.5.3.4 All data polled by Citect, no data Time Stamped. 7
2.5.3.5 Unsolicted Time Stamped Digital Input event transmitted to Citect: 7
2.5.3.6 Application Time Stamping of data 8
2.5.3.7 Application Time Stamping of individual data elements 8
2.5.3.8 User Time Stamping of complete rows 8
2.5.3.9 Interpretation of Hardware Timestamps 8
2.5.3.10 Unsolicted transmission of Change Of State data: 9
2.6 Special Requirements 9
2.7 Maximum Request Length 9
3 USER INTERFACE 10
3.1 Introduction 10
3.2 Driver Name 10
3.3 Boards Form 10
3.4 Ports Form 10
3.5 IO Devices Form 10
3.5.1 Address : # NameOfDevice 11
3.5.1.1 The critical RTU must be the first device in the IO Devices form 11
3.6 Pulldown lists Help 11
3.7 IO Device Variable Types 12
3.7.1 Formats and types 12
3.7.2 Event data types (alarm and trend) 13
3.7.3 Cicode functions to access timestamped event data 15
3.7.4 MOSCAD.dbf Entries 16
3.8 PROTDIR.DBF 16

3.9 Parameters and INI options 18
3.9.1 Standard Parameters 18
5.9.2 Driver Specific Parameters 18
3.9.1.1 General driver parameters - [moscad] 18
3.10 19
3.10.1 Format of configuration file 19
3.10.2 Sample of a configuration file 20
3.10.3 Sample of a CITECT.INI file 21
3.10.3.2 Extra notes 22
3.10.3.3 Cache and scheduling sections in the IO Device form apply only to IOServer caching. Not the drivers. 22
3.11 Using Gateway Redundancy 22
3.12 Remapping 23
3.13 Driver Specific Errors 23
3.14 Debug Messages 26
3.15 Stats Special Counters 26
3.16 Hints and Tips 27

2 Target Device(s) and Protocol

2.1 Introduction

This section defines the types of I/O Devices that are targeted by this driver.

2.2 Device Manufacturer

Motorola Communications Group

2.3 Device Definition

The device is a hardware system, consisting of a number of RTUs (Remote Terminal Unit) which communicate to the outside world via a Gateway. The Gateway abstracts communications so that all RTUs are communicated with using the same Application Programming Interface (API) calls regardless of the communication path (e.g. leased line, radio modem e.t.c).

The Gateway communicates to RTUs using the proprietary MDLC protocol over RS-485 and radio modem. The gateway has 3 serial ports for communications with the RTUs or the programming toolbox. Also the gateway has two network connectors (10Base-T and AUI), either of which can be used.

Communication with the gateway is possible using TCP/IP and a standard ethernet card.

Each RTU consists of a CPU module and various IO and communication modules.

2.4 Communications Method

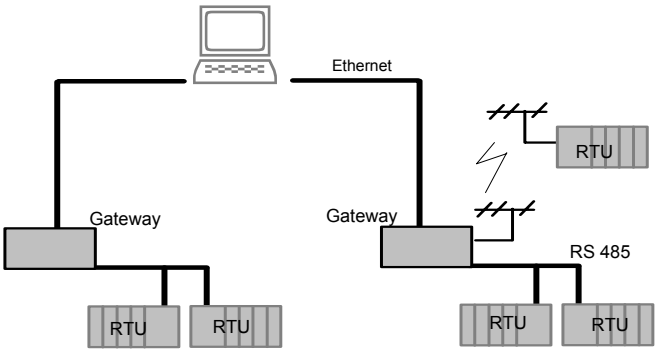
The Gateway sits on an ethernet network and communicates to the SCADA system using the TCP/IP protocol.

The Gateway can theoretically communicate to a total of 32768 RTUs at one time. The practical maximum is not specified but section 7.1 in the MOSCAD developers guide mentions that a minimum of 250 RTUs should be supported per gateway, and at least 16 gateways should be supported. Therefore the standard skeleton driver constants must MAX_UNIT must be at least 250, and MAX_CHANNEL must be at least 16.

From the MDLC API users point of view, each RTU consists of a user configurable number of tables (up to 128), each of user configurable dimensions (up to a maximum of 250 rows by 8 columns), and user configurable makeup (e.g. bit, bit, float or int, float, int). The columns are made up of only one data type.

The communication philosophy is client / server. A SCADA system is considered the **client** and the gateway the server. A client communicates to the server using one or more **channels** (TCP/IP sockets). It is recommended to use a different channel for each type of communications (commands, poll data, burst data and management data).

2.5 Communications/Hardware Configuration



2.5.1 Wiring Diagrams

Standard ethernet cabling for connecting Citect to Gateway.

Diagnostics cable for use with ToolBox Software:

Adaptor Cable Pinouts

Colour	*RJ45*	*DB25*	*DB9*
Blue	1	5	8
Orange	2	6	6
Black	3	8	1
Red	4	4	7
Green	5	7	5
Yellow	6	20	4
Brown	7	2	3
Grey	8	3	2

2.5.2 I/O Device Settings

Communications set up is a function of windows. The network card must be installed with support for TCP/IP loaded and the local IP address selected.

2.5.3 Software Setup

Set up the local IP address using standard Windows network installation tools from control panel.

There is a MOSCAD Programming Toolbox available to configure gateways and another to configure RTUs. The computer running the Toolbox uses an RS-232 connection, and can be connected to either a gateway or an RTU.

2.5.3.1 Gateway Software Configuration

The Gateway toolbox is used to modify the following four groups of parameters.

- TCP/IP Driver (IP address)
- MDLC Gateway configuration (MDLC Gateway, Site-id and Link-ids)
- MDLC Network configuration (system wide MDLC routers)

- MDLC Site table (system wide Site-id and Link-ids)

2.5.3.2 Gateway Mode of Operation Configuration

- If 1 Gateway is being used, the user should set the Gateway startup mode as “standalone”.
- For Gateway redundancy, the user should setup 1 Gateway as “Primary” mode and the second Gateway as “Standby” mode. This can be set using the Gateway ToolBox program.
- For “Primary” gateway operation, the user should set the gateway startup mode as “GW1”. For “Standby” gateway operation, the user should set the gateway startup mode as “GW2”.

2.5.3.3 RTU Software Configuration

The Gateway toolbox is used to modify the following four groups of parameters.

- SiteId (RTU address)
- RTU Site configuration (I/O cards, ports, etc)
- RTU Database configuration (RTU tables of data)
- RTU User Application configuration (RTU ladder logic)

The RTU Database configuration is used to hold either real I/O or user variables.
The RTU User Application configuration (ladder logic) is used to process variable and I/O data for monitoring and control purposes.
The amount of User Application configuration required is dependant on the communication strategy that is to be implemented. Sample RTU configuration is also included with (and is required for) the sample Citect project for this driver.

2.5.3.4 All data polled by Citect, no data Time Stamped.

Only user tables, for the data to be polled by Citect, need be configured in the RTU. The type configuration files (refer to Section 3.6), must also be edited to reflect these user tables.
In this case (the simplest possible case), no RTU User Application configuration (ladder logic) is required.

2.5.3.5 Unsolicted Time Stamped Digital Input event transmitted to Citect:

A user Time Stamped Digital Input table (this user table will be termed the TSDI table) must be configured, and linked to the required Digital Inputs. The column should be of type “TgDI” and should be linked as an “event” type, so that the RTU will place the Time Stamped Digital Input events onto the RTU event queue. Ladder Logic must be configured to extract a Time Stamped Digital Input event from the internal queue and place it into the system Primary Event Table. The state and time stamp data is then copied from the Primary Event Table to another user table (this user table will be termed the Buffered TSDI table). This updated row of the user table is then “burst” back to Citect. Refer to “MOSCAD Programming ToolBox – Special Functions” manual, Section “Event Driven Software”, for further background on the Primary Event table. Refer to “MOSCAD/MOSCAD-L Programming ToolBox for Windows (Version 5.0) – Users Guide” manual, Section “User Defined MDLC Communication”, Subsections “Data Burst Table” and “Burst Function” (Pages 309 and 310 in Version 5.0) for further background on “Bursting” data to Citect.
Note that in order to utilise the Burst function, the table holding the data must have a symbol defined for it, hence one must be allocated when creating the Buffered TSDI table. The Burst function relies on a system table, the “Site Table”, to define the Site ID and Link ID of the Gateway, so that data can be “burst” to Citect via the gateway.
The data column to be copied from the Primary Event table to the Buffered TSDI table, is “___Bit”. This column should be configured in the Buffered TSDI table as type “Bit”, and defined in the type configuration file (refer to Section 3.6) as “Bit”. The two timestamp columns in the Primary Event table, that have to be copied to the Buffered TSDI table, are “TmMost” and “TmLeas”. Both timestamp columns in the Buffered TSDI table are to be configured as type Real, but must be defined in the type

configuration file as TS1 for the column copied from “TmMost”, and TS2 for the column copied from “TmLeas”.
Note, if a Digital Input is being used in a Time Stamped Digital Input table, then it should not be duplicated in a standard Digital Input table. This may result in a value being shown in a timestamped digital alarm and a different (older) value still being shown in the Digital Input tag, as the Digital Input tag will not be updated with the Time Stamped Digital Input data.

2.5.3.6 Application Time Stamping of data

If the project is utilising application timestamping of data, then the application time stamping will be conducted every scan cycle (or second cycle, etc), and will be used to indicate the time at which this RTU scan acquired this data. User timestamping will only be accurate, at most, to the resolution of the RTU scan time. In comparison, the system timestamping of Digital Inputs is accurate to the millisecond, however Hardware timestamping places a large load on the RTU and should be used sparingly. The application timestamp will indicate the most recent time at which data has been acquired for this point, which may be a result of the RTU “bursting” an unsolicted response back to Citect, or the RTU responding to a Citect poll.

2.5.3.7 Application Time Stamping of individual data elements

Every column of data that is to be individually application time stamped is to be followed by two columns reserved for holding the time stamp data. Both timestamp columns are configured in the RTU as type Real, and are defined in the type configuration file (refer to Section 3.6) as TS1 for the column immediately following the data column, and TS2 for the next column. The content of these two columns is detailed in the manual “MOSCAD Programming ToolBox – Special Functions” , section “Event Driven Software”, under the heading “Event Driven Mechanism”. This section is detailing the Primary Event Table, used in the retrieval of Time Stamped Digital Events, but the Primary Event Table time stamp columns (listed as TmMost and TmLeas) can be set to the current Date/Time by calling the “Time” function in the users application code. These two columns are then copied to the required table locations to achieve user timestamping of data. The first timestamp column is copied from TmMost, and the second timestamp column is copied from TmLeas.

TmMost			
Day	Month	Year	Hour
1 Byte	1 Byte	1 Byte	1 Byte
1-31	1-12	0-99	0-23

TmLeas		
Minute	Second	Millisecond
1 Byte	1 Byte	2 Bytes
0-59	0-59	0-999

2.5.3.8 User Time Stamping of complete rows

As an alternative to user timestamping individual data elements, a complete row can be user timestamped. This is achieved by utilising the first two columns of a table as timestamp columns (ie both defined in table as “Real”, and defined in type configuration files as TS1 for the first column and TS2 for the second column). All data in that row will be time stamped with the Date/Time of this timestamp.

2.5.3.9 Interpretation of Hardware Timestamps

A decision has to be made whether a read to a TimeStamp tag of a TSDI is to represent the time at which an event last occurred for this point, or the last time at which data was acquired for this point.
The manner in which a timestamp (associated with a TSDI) is to be interpreted, will determine if the Buffered TSDI table needs to have the timestamp data continually updated by application timestamping, or if the timestamp is only updated by the hardware timestamping of the next event. In the first case, when the Time Stamped Digital Input event is being “burst” from this table, and in subsequent polls of this table, these timestamp columns will hold the time of the most recent event for this Digital Input. In the second case, the timestamp columns would be continually updated by application timestamping, in addition to the initial hardware timestamping, so that the timestamp columns will hold the time at which the data was acquired.
Note that, in the second case, the millisecond resolution of the hardware timestamping will be overridden by the scan time resolution of the application timestamping. However, this lower resolution will only be utilised in the polling algorithm and will only be used to update graphics; the alarm system will utilise the initial timestamp, with its millisecond resolution, that has been “burst” to Citect. The

advantage of the second case is that the timestamps for the hardware based Time Stamped Digital Inputs will have the identical functionality of the application based timestamped data, ie the timestamp relates to the time at which the data was acquired. In this way, a tag read to the timestamp of this data will display the last point in time at which this data was acquired, either through unsolicited response or polling. This mechanism will also ensure that a timestamp value is obtained for a timestamp tag read, even if an event has not yet occurred since Citect startup.
If TimeStamp tags (that is reading the TimeStamp value) will not be used in the project for these points, then application timestamping of these hardware timestamped points is not required.

2.5.3.10 Unsolicted transmission of Change Of State data:

Ladder Logic must be configured to test the COS flag on each row of each table, and if the flag indicates that an element in that row has changed its value, then the row must be "burst" back to Citect. Refer to "MOSCAD/MOSCAD-L Programming ToolBox for Windows (Version 5.0) – Users Guide" manual, Section "User Defined MDLC Communication", Subsections "Central To RTU Data Transfer" (Page 293 in Version 5.0) for further background on the COS flag.
Note that in order to utilse the COS flag, the table holding the data must have a name defined for the flag, and hence one must be configured by editing that table.

2.6 Special Requirements

We will need to ship the Winsock.DLL or alert users to the need for it.

The following information concerns the usage of the MOSCAD MDLC API.

Before running the driver make sure that the configuration files ("<mdlc_par>.cfg", where mdlc_par is your file name. The path of this file must be set up in Citect.ini file as well. Refer to section 5.10) is present in the local directory and properly describe your MOSCAD system. To work properly, the API routines need information about the MOSCAD system and about the RTU types (communication table format). This information must be provided via two ASCII files:

<mdlc_par>.cfg - This file describes the format of the different existing RTU types (and future but currently non-existing RTU types) and type of each RTU in the MOSCAD system. There is a description of the communication tables format for each of the RTU types. The format defines the number of columns in a table, and the type of each column. A column can be either "Bit", "Val", "Float", "C_Bit", "TS1" or "TS2"; with Bit being for digitals, Val for 16 bit integers, Float for reals, C_Bit for eight columns of bits being compressed into one byte, and TS1 and TS2 used collectively for Time Stamping. Note that a TS1 column must be immediately followed by a TS2 column.

In this driver, there are two methods of timestamping data: individually or on a "per row" basis. To individually timestamp a data element, the data column in the table should be immediately followed by a TS1 and a TS2 column to hold the timestamp for that data element (eg Bit TS1 TS2). In order to collectively timestamp an entire row, the first column should be of type TS1 and the second column should be of type TS2, all subsequent columns in the row will be timestamped by this value.

2.7 Maximum Request Length

2048 bits

Client to Gateway Sync RTUs Code Data Block
Data Block Type Sync RTUs Code
Block Number
Block Status=0
Data Length
User's Data
Coordinates
None.

Gateway to Client Response Data Block
Data Block Type Sync RTUs Code
Block Number
Block Status
Data Length
User's Data
Coordinates
None.

3 User Interface

3.1 Introduction

This section defines how the user will see the driver. This relates directly to how the Citect forms need to be filled out and any special INI options. For the kernel, the debug trace messages and the Stats.Special counters are documented.

3.2 Driver Name

MOSCAD

3.3 Boards Form

Field	Default	Allowable values
Board Name	This field is user defined, and is not used by the driver.	
Board Type	TCPIP	
Address	0	
I/O Port	BLANK	
Interrupt	BLANK	
Special Opt	BLANK	
Comment	This field is user defined and is not used by the driver.	

3.4 Ports Form

(A port maps to a gateway.)

Field	Default	Allowable values
Port Name	This field is user defined and is not used by the driver.	
Port number	Must be unique, but is not used by the driver	
Board name	Refers to the board previously defined in 'boards' form.	
Baud rate	BLANK	
Data bits	BLANK	
Stop bits	BLANK	
Parity	BLANK	
Special Opt	PLC's IP address*	
Comment	This field is user defined and is not used by the driver.	

- The IP address must use the following format: **-Ia**

Where: **a** is the destination IP address in standard Internet dot format

Example: **–I280.9.21.60**

There may or may not be a hot backup gateway. If there is a hot backup defined then the user must follow the instructions detailed in section 2.5.3.2 Gateway Mode of Operation Configuration

3.5 IO Devices Form

(An IO Device will map to an RTU.)

Field	Default	Allowable values

Name	This field is user defined, and is not used by the driver.
Number	Must be unique, but is not used by the driver.
Address	# NameOfDevice
Protocol	MOSCAD
Port name	Refers to the port previously defined in 'ports' form.
Comment	This field is user defined and is not used by the driver.

3.5.1 Address : # NameOfDevice

where:
is the RTU address between 1 and 32768
NameOfDevice is an optional field that associates this unit with the parameters defined in the Citect INI file defined by the section '[*NameOfDevice*]'.
If this optional field is not utilised, then this unit will just use the values defined by the global INI parameters defined by the [MOSCAD] section.
Note that for units that require identical configuration of INI parameters, they can all reference the same *NameOfDevice,* and hence be associated with the same INI section.

3.5.1.1 The critical RTU must be the first device in the IO Devices form

3.6 Pulldown lists Help

The following entries should be included in the Citect HELP.DBF spec file.

TYPE	DATA	FILTER
PROTOCOL	**MOSCAD**	
BOARD	**MOSCAD**	

3.7 IO Device Variable Types

3.7.1 Formats and types

IO Device Type	Citect data format	Citect data types	Description/Special Usage/Limitations/ Valid Ranges
Bit	Bz.y.x	DIGITAL	Read / Write
Compressed Bit	CBz.y.x	DIGITAL	Read Only
Analog	Az.y.x Az.y.x.w	INTEGER DIGITAL	Read / Write Read Only – Single bit of analog
Float	Fz.y.x	REAL	Read / Write
TimeStamp for data element (time in seconds since 01/01/1970 in UCT format)	TSz.y.x	LONG	Read Only The z.y.x address of a timestamp, when constructing a Citect tag, will be the z.y.x address of the data element that the timestamp corresponds to. The Timestamp will physically reside in the first two columns for row timestamped data or in the following two columns for individually timestamped data
RTU Time (time in seconds since 01/01/1970 in UCT format)	RTUTime	LONG	Used to read RTU Date / time Read Operations on this tag will read the value from diver cache. Write Operations on this tag will update the value held in driver cache (this prevents continuous device requests from a screen tag) Use Cicode functions for displaying time/date.
Synchronise RTU time with Citect	SynchRTUTime	DIGITAL	Write Only Driver will always return zero if user does try to display tag
Force Poll of RTU	ForcePoll	DIGITAL	Write Only
Force COS Poll of RTU	ForceCOSPoll	DIGITAL	Write Only
RTU Status	RTUStatus.[< STALE, EPOLL, IPOLL, POLL, AV, INUSE, CRIT>]	DIGITAL DIGITAL	Read Only – status of RTU tag is assigned to
Broadcast	BCast *a.b.c*	INTEGER	Write Only *a* = TargetRow

			b = TestData *c* = TestRow Argument = TargetData If TestRow of Table 0 of an RTU holds the value TestData, then TargetRow of Table 0 of that RTU will be loaded with TargetData *(Note that this format is only applicable for 'virtual units' – memory or disk PLCs. Refer to note below on virtual units)*
Cache Age	CAGEz.y.x	REAL	Cache Age
Tag Status	STATUSx.y.x	REAL	Tag Status

Where:

z	the table number in the RTU data base	0..127
y	the row number in the table	0..249
x	the column number in the table	0..7
w	The selected bit of an analog	0..15

Virtual unit A virtual unit is a unit that is defined in the Devices form as a memory or disk PLC, and has special tags assigned to it in order to interact with the driver. In this case, a virtual unit is created for checking the status of real units, so that the status tag will not be #COM'd when the unit is offline. A virtual unit is also created for the broadcast operation, as this operation effects multiple units, and hence a broadcast tag is not related to just one unit. There are INI parameters to distinguish the unit address as virtual units, refer to Section 5.9.2 for details.

3.7.2 Event data types (alarm and trend)

This section is concerned with the processing of time stamp data. A data element in an RTU table is timestamped by the RTU application configuration placing time stamp data in the two columns following the data element. The first column is of type TS1 and the second column is of type TS2. For further information refer to Section 3.5.3.2. When a data element and its time stamp are received by the driver, the data and the time stamp are placed in the cache. This means that the data and the time stamp are available for normal tag read operations.
In addition to this, the data and its associated time stamp, as well as other relevant information, is placed onto an event queue. This queuing system, and the interrogation of the queue through Cicode, was developed for the KINGFISHER driver.
Each event queue item contains the following information:

TIMESTAMP in Seconds Since 01/01/70 (In Universal Coordinate time) – provides Date and Time
MILLISECONDS Since Midnight – provides Millisecond Accuracy for digital alarms
VALUE
VALUE TYPE (2=Bit, 4=Analog or 5=Float)
TAG NAME

Refer to 5.7.3 for further details.

The event data received may, or may not, contain timestamps. Either way the value contained in the event data is used to update the driver's cache. If the event data is time-stamped, then the driver will check to see if the event data is required for Time Stamped Alarms or for Trends. If it is, then it will pass the event data onto the driver's event que, which will then in turn be interogated by dedicated Cicode, that will process the event data and update the associated Alarm or Trend.
The project must initiate the dedicated Cicode if event driven alarms or trends are to be used. This is accomplished by defining a *Parameters* form (from the *System* menu) as follows:

Section Name:	Code
Name:	Startup
Value:	Tasknew("RTUEventQueProcessing",0)

To simplify the matching process of determining the trend tag associated with an incoming time stamped unsolicited response, the following naming convention is used. For Periodic Trends, determine the tagname that corresponds to the I/O point from which the event originated, and suffix the tagname with "T_". For instance, if B1.0.0 generates an event, and has a tag PUMP_20_RUNNING defined for it, then the corresponding trend tag should be named T_PUMP_20_RUNNING. By using the tag PUMP_20_RUNNING as the trend tags expression, the trend will continue to trend the value held in cache in between events. Note that the event will be inserted into the trend at the time in which the event occurred, not the time at which the event was received. The dedicated Cicode will then redraw the trend (with the new value) from the time of the event up until the present time. Similarly, for a Periodic Event trend, the trend tagname should be suffixed with "P_". For example, P_PUMP_20_RUNNING.
In the future, Event trends will be supported by the "E_" suffix, as in E_PUMP_20_RUNNING.
Trends for analog received events are not currently supported, but will be implemented in the same manner as that described for digital received events, once they are supported.
Only use standard templates when constructing trend displays to be used by event driven trends. If a "zoomtrend" is to be used, then set the [DNP] parameter *ZoomTrendInUse=1.*
It is still possible to use normal Citect trends. By using a trend name that does not follow the above naming convention(eg simply using the tagname as the trend tagname), then the trend will be of the value held in driver cache and will be relative to Citect time. This implies that changes in value will relate to the time at which the data was received, rather than when the event occurred.
Note that the trending system does not support the importing of data in millisecond resolution, hence the main use of trending time stamped data is in a system where there is unacceptable, or variable, delay between when the RTU scans its I/O and when Citect receives and timestamps the data. If this is not an issue, then normal trending can be used, with Citect trending the data held in the driver cache. Note that the driver cache will be updated in between polls if an unsolicited response is received.
For each time stamped digital alarm configured in the RTU to be "burst" to Citect, two tags are to be defined in the following format. Determine the tagname that corresponds to the I/O point from which the alarm originated, and suffix the tagname with "A_". For instance, if B1.0.1 generates an event, and has a tag PUMP_20_TRIPPED defined for it, then the corresponding Time Stamped Alarm tag should be named A_PUMP_20_TRIPPED. The *variable* tag for the alarm should be suffixed by "V_", as in V_PUMP_20_TRIPPED, and the *timer* tag should be suffixed by "S_", as in S_PUMP_20_TRIPPED. Both the "V_" and "S_" tags should be virtual tags defined for a memory PLC, with the "V_" tag having a D*n* address and of type Digital, and the "S_" tag having a L*n* address and of type Long.
Cicode will extract events from the driver's event queue, and if the data is digital, write the time stamp to the "S" tag, and then write the value to the 'V" tag (if they have been configured). This will trigger the Time Stamped Alarm tag, which will then display with the state and time of occurrence as timestamped in the RTU.
As the Alarm system polls tags for changes in value, but events are received compressed in time, then any consecutive events **for the same point** must be delayed by an interval equal to or greater than the polling time used by the alarm system. If this is not assured, then there is a risk of alarms being missed as the events will be fed to the alarm server quicker than it is polling for changes. The [ALARM] parameter *ScanTime* allows the user to control the actual trend system polling time. The driver reads this value, and enforces its own minimum of 1 second, so that it can enforce this delay between consecutive events of the same point. The "Citect Info" function can be used to determine the "Maximum time between code execution (cycles)" for Time Stamped Alarms – CitectInfo("Stats","High Res. AI",1). The *ScanTime* parameter must be adjusted so as to ensure that the actual alarm scan time does not exceed the expected alarm scan time.

Note also that the timestamp can be viewed in one of two forms, Date/Time or Time/Millisecond. The form to be used is defined by the [ALARM] parameter *HresType*. Setting *HresType=4* defines Date/Time format, and setting *HresType=7* defines Time/Millisecond format. A format must be selected in order for device timestamped alarms to work, so an error will be generated if it is not defined in the CITECT.INI file. The [ALARM] parameter *HighResOff* should be set to 1 (*HighResOff=1*) to force millisecond accuracy when alarms are turned off, if Time/Millisecond format is being used. An appropriate alarm category must also be defined to correctly utilise the selected format.

Refer to the sample project for examples of event driven Trend tags and event driven Time Stamped Alarm tags.

3.7.3 Cicode functions to access timestamped event data

GetRTULogStatus()

Purpose: To indicate whether there are records available for extraction from the internal driver event cache.
Return Value: A value representing the number of records remaining in the driver cache (LONG).
Parameters: NONE

GetRTUNextEvent()
Purpose: To remove the event record from the front of the internal driver event cache, and make it available for field data extraction using GetRTUEventField.
Return Value: A handle to the first event OR -1 if no events in internal queue (LONG)
Parameters: NONE

GetRTUEventField(hEvent, fieldNumber)
Purpose: To obain a field from the current extracted event record.
Return Value: Selected event record field as a string (STRING)
Parameters: hEvent - event handle (LONG) Range 1-65,535
fieldNumber - field number (LONG) Range 1-5

Field Number	Field Name	Field Description	Cicode Numerical Type. (Cicode to convert field from STRING to the listed type)
1	TIMESTAMP	Time in seconds since 1970, in Universal Coordinate Time	INT
2	MILLESECONDS	Milliseconds since midnight – additional accuracy for optional use in Time Stamped Digital Alarms.	INT
3	VALUE	Value of event	Bit INT Analog INT Float REAL
4	VALUE TYPE	Used to determine data type of value (2=Bit, 4=Analog or 5=Float)	INT
5	TAG NAME	Tag Name	STRING

EmptyEventQue()
Purpose: To empty the internal driver event queue.
Return Value: Always returns 0.

3.7.4 MOSCAD.dbf Entries

Notes on driver dbf.

This driver is unusual in that the user can configure the structure of the RTU communication tables (Citect data types). Each table communication tables can have a number of columns between 1 and 8 and a number of rows between 1 and 250. In addition, the data type make up of the rows can also vary from table to table (eg int, int, float, bit or float, int, bit , bit, bit, bit). This variability makes it difficult for the Citect compiler to block requests correctly. It is possible for the driver to block along columns, so this method will be permitted by the driver where possible.

Blocking along columns is used for types Analog (A), Float (F) and TimeStamp(TS) because these types have a bit width greater than 8 bits, which is the minimum size that the IO server will block correctly (1 bit blocking does not work properly). This blocking approach relies on continuity of address space (assured in a table column) and that the IOServer will not request an address larger than the largest address (row) the user uses as a tag for a particular UnitType (column) i.e. any “incorrect address” errors will be a result of the user adding an incorrect tag rather than as a result of the IO server’s optimisation.

The Bit type (B) cannot be blocked at all, because no blocking along rows is possible (due to type variability) and no blocking along columns is possible (as it cannot be guaranteed that there will be multiples of 8 rows).

From this scheme then:

Bits 0..7 of the UnitType field are reserved for the base type (A, F… etc)
Bits 8..15 are reserved for the table number where required
Bits 16..23 are reserved for the column number where required

The driver has access to information of the makeup the columns and rows from the API, and so can optimise the requests from Citect.
Note that since the driver will block along columns rather than rows (i.e. against the natural order the API uses), the writing of multiple data items e.g. strings will result in one Citect write request being split into multiple API requests to write individual data items.

TEMPLATE	UNIT TYPE	RAW TYPE	BIT WIDTH	LOW	HIGH	COMMENT
A%<8(0,1,128).%U(0,0,249).%<16(0,0,7)[.%u(0,0,15)]	1	1	16	0	0	Analogues
F%<8(0,1,128).%U(0,0,249).%<16(0,0,7)	2	2	32	0	0	Floats
TS%<8(0,1,128).%U(0,0,249).%<16(0,0,7)	3	4	32	0	0	TimeStamp
CB%<8(0,1,128).%U(0,0,249).%u(0,0,7)	4	8	8	0	0	Compressed Bits
B%<8(0,1,128).%<U(0,0,249)%*8.%<16(0,0,7)	5	0	1	0	0	Bits
RTUTime	6	4	32	0	0	RTU time in seconds
RTUStatus[.%u<STALE,EPOLL,IPOLL,POLL,AV,INSUE,CRIT>]	7	1	16	1	32768	Online status of RTU
SynchRTUTime	8	0	1	0	0	Synchronise RTU time to Citect time
BCast%<8(0,0,249).%U(0,0,65535).%<16(0,0,249)	9	1	16	0	0	Broadcast qualified control message (BCASTTargetRow.TestData.TestRow)
ForcePoll	10	0	1	0	0	Force a poll (non-COS) of unit
ForceCOSPoll	11	0	1	0	0	Force a COS (Change Of State) poll of unit
CAGE%<8(0,1,128).%U(0,0,249).%16(0,0,7)	12	2	32	0	0	Cache age
STATUS%<8(0,1,128).%U(0,0,249).%<16(0,0,7)	13	2	32	0	0	Tag status

3.8 PROTDIR.DBF

MaxBits calculated as

Max size of Moscad receive buffer = 20480 * 8 = 163840 bits

Max size of Moscad table row = 8 columns * sizeof(float) = 8 * 32 = 256 bits.

Max no of Max Size rows that can be read in one request from Moscad = 163840 / 256 = 640 rows.

Max size of Citect request of floats = 2048 / 32 = 64 items (means driver requests 64 rows because of blocking along columns).

Therefore MaxBits is maximum possible for Citect ie 2048 bits.

TAG	FILE	BIT_BLOCK	MAX_LENGTH	OPTIONS
MOSCAD	MOSCAD	2048	2048	0x10cb[*1]

[*1] **- Supports Digital, Integer, Long, Real, Byte and blocking on 8 bit boundaries.**

3.9 Parameters and INI options

3.9.1 Standard Parameters

Block	256	
Delay	0	(Citect V5 onwards does not require a delay in the driver)
MaxPending	32	
Polltime	0	
Timeout	3000	(Milliseconds)
Retry	1	
WatchTime	75 (Seconds)	

5.9.2 Driver Specific Parameters

3.9.1.1 General driver parameters - [moscad]

Parameter	Default	Allowable values	Description
Channel	7	??	
Multiplexed	1	0 or 1	Enable redundancy at a unit level
MaxTimeouts	3		Count
ConnTimeout	15000		Ms. Connection timeout before a connection error is reported
TimeOutExt	0		Global timeout extension, see per channel parameters
ReportUnknownAsAvail	1	0 or 1	If set to 1, then connection availability is TRUE if the units status is NOT OFFLINE. If set to 0, then connection availability is TRUE if the units status is ONLINE.
DatabaseLog	0	0 or 1	This parameter enables a database file of all received events (that have a Trend Tag or TimeStamped Alarm Tag defined) to be logged. This functionality requires the Devices form to be completed with an entry labelled RTULog. The Format in the form must correspond to that used in the Cicode program (refer to a sample project for this format). The file will reside in the location described by the File Name field entry in the Devices form.

Parameters per port – [portname]
NB: Where “channel” exists, read this as the same as a “Port”

Parameter	Default	Allowable values	Description
SetupFile	moscad.cfg		Setupfile for the channel
GatewayMode	1	1 or 2	This parameter instructs the driver to treat the startup mode of each channel. If the GatewayMode is set to 1, the driver will put the Gateway into “Active” mode on Citect startup. If the gatewayMode is set to 2, the driver will put the Gateway into “Inactive” mode on Citect startup and this will prevents the gateway from data exchange with the RTUs.
Channel	7	???	Channel type setting or uses global value

Polled	1	0 or 1	1 polled, 0 not polled. This is a configuration parameter which allows you to poll different tables in different RTU. It must be set in the moscad configuration file.
Multiplexed	1	0 or 1	Channel setting or uses global value
Tick	500		Ms. Poll time if enabled.
NoPollPeriod	10000		Ms. This is the INI parameter that controls the No Poll period in milliseconds. This setting will gurantee that during the NoPollPeriod there is no polling request sent to RTU on each channel.
DebugLevel	0	0 or 1	1 for individual channel debug traces
Timeout	3000	1 to 32767	Ms. Message timeout
TimeoutExt	0		Ms. Extension time on the timeout.
NoStatusCheck	0	0 or 1	1 to disable status checks on coms to the gateway

3.10

To work properly, the MOSCAD driver need information about the MOSCAD system and about the RTU types (communication table format). This information must be provided via an ASCII files, called configuration file. The configuration files (“<mdlc_par>.cfg”, where mdlc_par is your file name) can be any name and anywhere in your local driver provided the path of this file is set up in Citect.ini file properly. To set up the path in Citect INI file, simply put SetupFile=”Your File Path\\<mdlc_par>.cfg “ under [MOSCAD] section in INI file. Note, use double back slash as delimiter. Please see the example in 5.10.3 for details.

<mdlc_par>.cfg - This file describes the format of the different existing RTU types (and future but currently non-existing RTU types) and type of each RTU in the MOSCAD system. There is a description of the communication tables format for each of the RTU types. The format defines the number of columns in a table, and the type of each column. A column can be either “Bit”, “Val”, “Float”, “C_Bit”, “TS1” or “TS2”; with Bit being for digitals, Val for 16 bit integers, Float for reals, C_Bit for eight columns of bits being compressed into one byte, and TS1 and TS2 used collectively for Time Stamping. Note that a TS1 column must be immediately followed by a TS2 column.

In this driver, there are two methods of timestamping data: individually or on a “per row” basis. To individually timestamp a data element, the data column in the table should be immediately followed by a TS1 and a TS2 column to hold the timestamp for that data element (eg Bit TS1 TS2). In order to collectively timestamp an entire row, the first column should be of type TS1 and the second column should be of type TS2, all subsequent columns in the row will be timestamped by this value.

3.10.1 Format of configuration file

Format of line:

<label>:<id>:<parameter 1>;<parameter 2>[<arglist>];...<parameter n>:<text command>

where:

label is a text string without space and not starting with a number. eg Table0, Table_0.
id is a number 0-65536 used to identify the entity.

parameter 1.. n
for tables are:

polled this table will be polled.

rows <no> is the number of row in the table.
coloumns <no> is the number of coloumns.

for units are:

polled this unit will be polled.
period <no ms> is the number of ms between polls.
ratio <ratio> is the number of event polls between integrity polls.
tables <label label ...> is a list of table label defined before.
the unit label configured in the unit.

off for both tables and unit:
removes the unit/table from the list without causing syntax problems.

text command
for tables:
this command defines the column type in the table.
<type> <type> <type>

where:

type can be:
"real"
"discrete"
"analog"

for units:

this command defines the path and file name of the persistent cache of the unit

Note in standard unix form the line can traverse several line in the file by making the last charactor of the line prior to the <CR> a \.
Any charactors to the right of a # to the end of the line is comment.

3.10.2 Sample of a configuration file

```
# This is a sample of configuration file
#
# Table Configuration:
#
#<label> | : | <id> | : | < para1> | ; | < para 1> | ; | < para 1> | : | <Text Command>
#-----------------------------------------------------------------------------------------------------------------------
Table1    :  1    :  Polled    ;  Rows 4    ;  Columns 1    :  Discrete
Table2    :  2    :  Polled    ;  Rows 4    ;  Columns 1    :  analog
Table3    :  3    :  Polled    ;  Rows 4    ;  Columns 1    :  real
Table4    :  4    :            ;  Rows 4    ;  Columns 1    :  discrete real real
Table5    :  5    :            ;  Rows 4    ;  Columns 3    :  analog real real
Table6    :  6    :            ;  Rows 4    ;  Columns 3    :  real real real
Table7    :  7    :            ;  Rows 4    ;  Columns 3    :  real real discrete
Table8    :  8    :            ;  Rows 4    ;  Columns 3    :  real real discrete analog real
#
# RTU Configuration:
#
#<label> | : | <id> | : | < para1> | ; | < para2> | ; | < para3> | ; | < para4> | : | <Text Command>
#-----------------------------------------------------------------------------------------------------------------------
RTU1      :  1    :  Ratio 2   ;  Polled    ;  Period 6000   ;  Tables Table1\
                                                               Table2 Table3\
                                                               Table4 Table5\
                                                               Table6 Table7\
                                                               Table8          :  d:\tmp\rtu1.dat
RTU2      :  2    :  Ratio 2   ;  Polled    ;  Period 5000   ;  Tables Table1\
                                                               Table2 Table3\
                                                               Table4 Table5\
                                                               Table6 Table7\
                                                               Table8          :
```

3.10.3 Sample of a CITECT.INI file

```
            [MOSCAD]
            !Setup Parameters for Citect Test Rig
!
            !There are two discrete links used by Citect Test Rig
!           Rtus 1-2 on link1
!           Rtus 1 on link2
!

!Master Station or Gateway name

3.10.3.1.1  Station=STATIONABC
! The Master station is the one responsible for polling the moscad gateways
!Citect timeout and retry
            Timeout=10000
            Retry = 2


!Burst, Send and/or Receive Channel
!           0 NORMAL - no Burst
!           1 SEND
!           2 RECEIVE
!           4 BURST

3.10.3.1.2 Channel=7
! Channel = 7 sets the mode of the channel.
Since it follows the station name definition and preceeds any specific channel.(e.g. STATIONABC)  It
is the default channel mode for any following channel definitions, 7 = 4 + 2 + 1
!Other Parameters
!           DebugLevel
!           MaxTimeouts
!           ConnTimeout
!           RedundancyTick
!           Tick
            debugstr=PORT3_BOARD1 all

[STATIONABC0]
!Channels is not capable of fullduplex comms

3.10.3.1.2.1    Multiplexed = 1
! Multiplexed This permits multiple requests on a single channel.
    Timeout=30000
    NoPollPeriod=2000

!Rtus 1 on link1 configuration

3.10.3.1.2.1.1.1.1 SetupFile="d:\\CTDDK5\\VC6\\Drivers\\moscadip\\moscad1.cfg"
[STATIONABC1]
!Channels is not capable of fullduplex comms
    Multiplexed = 0
    Timeout=30000
    NoPollPeriod=2000

!Rtus 2 on link2 configuration

3.10.3.1.2.1.1.1.2 SetupFile="d:\\CTDDK5\\VC6\\Drivers\\moscadip\\moscad2.cfg"
[STATIONABC2]
!Channels is not capable of fullduplex comms
```

Multiplexed = 1

Timeout=3000
NoPollPeriod=7000
!Rtus 2 on link2 configuration

3.10.3.1.2.1.1.1.3 SetupFile="d:\\CTDDK5\\VC6\\Drivers\\moscadip\\dnp.cfg"
========== End of INI ----------

3.10.3.2 Extra notes

Polling definitions are defined in cfg. The cfg captures the MOSCAD unit definitions and table definitions. This parameter format is not supported by "ini" style data entry where a parameter occupies one line.

To have different tables on the same unit polled at different frequencies is not supported. The Full (integrity) poll, Change of State (COS) Poll and burst modes allow for optimising through put.
Note: All RTU can be in one .cfg file as specified by the MOSCAD(user spec).doc. You only need use different setup files (*.cfg) if you need to have multiple channels eg

```
[MOSCAD]
station = BASE
setup = "...defaultChannel.cfg"
[BASE0]
setup = "... firstChannel.cfg"
[BASE1]
setup = "... secondChannel.cfg"
```

where the intention is to use firstChannel to get to data for units, UNIT1 - UNIT20 say, and also use secondChannel to get to data for units, UNIT21 through UNIT30 say. So the idea is flexibility. However you can use one one default configuration file, as for defaultChannel.cfg in the above example.

3.10.3.3 Cache and scheduling sections in the IO Device form apply only to IOServer caching. Not the drivers.

Please be aware that the MOSCAD driver is a Front End / Back End (FE/BE) driver. Which means that there is an internal driver cache - separate from the IOServer cache.
Note: The parameters in the .cfg file pertain to the BE of the driver. I.e. the driver has a back end polling engine which will perform the regular polling tasks and processing of burst data regardless of data demand from the IOServer.

4. The channel will transmit data in one of the following means. Full (Integrity) Poll, COS Poll and burst modes. The combination of these will allow the engineer to optimise the bandwidth/throughput for the channel.

3.11 Using Gateway Redundancy

To use Gateway redundancy, the user must set the Gateway mode parameter in the Citect.ini file:

Eg.

```
[MOSCAD]
Station=CitectChannel

[CitectChannel0]
GatewayMode=1 !treat gateway as "active" mode

[CitectChannel1]
GatewayMode=2 !treat gateway as "inactive" mode
```

[Refer to section 5.9.2 Driver Specific Parameters for details of the different startup modes]
[Refer to section 2.5.3.2 Gateway Mode of Operation Configuration for details]

3.12 Remapping

Remapping is not required or supported.

3.13 Driver Specific Errors

Driver Error Code (Hexl)	Mapped to (Generic Error label)	Meaning of Error Code
0x1000	GENERIC_ERROR	generic OK
0x1031	GENERIC_GENERAL_ERROR	general error
0x1032	GENERIC_UNIT_WARNING	busy error
0x1033	GENERIC_UNIT_WARNING	ready
0x1034	GENERIC_UNIT_WARNING	item is empty
0x1035	GENERIC_UNIT_WARNING	item is full
0x1036	GENERIC_INVALID_DATA_FORMAT	size problem
0x1037	GENERIC_INVALID_DATA_FORMAT	sign problem
0x1038	GENERIC_UNIT_WARNING	port problem
0x1039	GENERIC_UNIT_WARNING	code problem
0x103A	GENERIC_INVALID_DATA	data problem
0x103B	GENERIC_INVALID_DATA	failure problem
0x103C	GENERIC_UNIT_WARNING	time problem
0x103D	GENERIC_UNIT_WARNING	lock problem
0x103E	GENERIC_UNIT_WARNING	not found
0x103F	GENERIC_UNIT_WARNING	exists
0x1040	GENERIC_INVALID_DATA	illegal type
0x1041	GENERIC_UNIT_WARNING	illegal rate
0x1042	GENERIC_UNIT_WARNING	erasing flash failed
0x1043	GENERIC_UNIT_WARNING	prgrm flash failed
0x1044	GENERIC_UNIT_WARNING	illegal mode
0x1045	GENERIC_INVALID_DATA	illegal format
0x1046	GENERIC_UNIT_WARNING	abort
0x1047	GENERIC_UNIT_WARNING	non existing loop
0x1081	GENERIC_UNIT_WARNING	Communication Error
0x1082	GENERIC_CHANNEL_OFFLINE	Application Error
0x1083	GENERIC_UNIT_WARNING	Secondary mode Error
0x1084	GENERIC_UNIT_WARNING	duplicate entries
0x1085	GENERIC_UNIT_WARNING	illegal name
0x1086	GENERIC_INVALID_DATA	Invalid file version
0x1087	GENERIC_NO_MEMORY	Failed to allocate memory.
0x1088	GENERIC_BAD_PARAMETER	Problem accessing the definition files.
0x1089	GENERIC_UNIT_WARNING	Error in the definition files.
0x108A	GENERIC_INVALID_DATA	Non existing site ID.
0x108B	GENERIC_GENERAL_ERROR	The API internal data structure is corrupted
0x108C	GENERIC_INVALID_DATA_FORMAT	Invalid row
0x108D	GENERIC_INVALID_DATA_FORMAT	Invalid column
0x108E	GENERIC_INVALID_DATA_FORMAT	Table number error - out of range or table is not a valid communication table
0x108F	GENERIC_INVALID_DATA_FORMAT	Data element coordinate error - Site ID or table number unknown.
0x1090	GENERIC_GENERAL_ERROR	Not Currently Used (ORDER)
0x1091	GENERIC_INVALID_DATA	Received only part of the requested length
0x1092	GENERIC_UNIT_WARNING	There are warning indications.
0x1093	GENERIC_UNIT_WARNING	Error in the message structure. The data entities inside the API message buffer should be of complete RTU rows, columns or tables (or a single one).
0x1094	GENERIC_UNIT_OFFLINE	The current MDLC block returned an error status - could not parse the block.
0x1095	GENERIC_BAD_PARAMETER	Not Currently Used (LAST ENTITY)
0x1096	GENERIC_BAD_PARAMETER	Illegal IP address or IP address format

0x1097	GENERIC_INVALID_DATA	Not Currently Used (USED)
0x1098	GENERIC_INVALID_DATA	Error returned from a BSD socket library routine
0x1099	GENERIC_INVALID_DATA	API internal MOSCAD field system description does not match the MOSCAD field.
0x109A	GENERIC_INVALID_DATA	Data entity element type error
0x109B	GENERIC_INVALID_DATA	The hash table is full
0x109C	GENERIC_INVALID_DATA	Parameter error.
0x109D	GENERIC_INVALID_DATA	Name is too int32.
0x109E	GENERIC_GENERAL_ERROR	The entry requested was not found

The following entries should be included in the Citect PROTERR.DBF spec file.

PROTOCOL	MASK	ERROR	MESSAGE	REFERENCE	ACTION	COMMENT
MOSCAD		0x1031	ERR_ERROR			General Error
MOSCAD		0x1032	ERR_BUSY			Busy Error
MOSCAD		0x1033	ERR_READY			Ready
MOSCAD		0x1034	ERR_EMPTY			Item Is Empty
MOSCAD		0x1035	ERR_FULL			Item Is Full
MOSCAD		0x1036	ERR_SIZE			Size Problem
MOSCAD		0x1037	ERR_SIGN			Sign Problem
MOSCAD		0x1038	ERR_PORT			Port Problem
MOSCAD		0x1039	ERR_CODE			Code Problem
MOSCAD		0x103A	ERR_DATA			Data Problem
MOSCAD		0x103B	ERR_FAIL			Failure Problem
MOSCAD		0x103C	ERR_TIME			Time Problem
MOSCAD		0x103D	ERR_LOCK			Lock Problem
MOSCAD		0x103E	ERR_NOT_FOUND			Not Found
MOSCAD		0x103F	ERR_EXIST			Exists
MOSCAD		0x1040	ERR_TYPE			Illegal Type
MOSCAD		0x1041	ERR_RATE			Illegal Rate
MOSCAD		0x1042	ERR_FLASH_ERASE			Erasing Flash Failed
MOSCAD		0x1043	ERR_FLASH_PRGRM			Prgrm Flash Failed
MOSCAD		0x1044	ERR_MODE			Illegal Mode
MOSCAD		0x1045	ERR_FORMAT			Illegal Format
MOSCAD		0x1046	ERR_ABORT			Abort
MOSCAD		0x1047	ERR_LOOP			Non Existing Loop
MOSCAD		0x1081	ERR_COMM			Communication Error
MOSCAD		0x1082	ERR_APPLIC			Application Error
MOSCAD		0x1083	ERR_SECONDARY			Secondary Mode Error
MOSCAD		0x1084	ERR_DUP			Duplicate Entries
MOSCAD		0x1085	ERR_NAME			Illegal Name.
MOSCAD		0x1086	ERR_VERSION			Invalid File Version.
MOSCAD		0x1087	ERR_MEM			Failed To Allocate Memory.
MOSCAD		0x1088	ERR_FILE			Problem Accessing The Deffinition Files.
MOSCAD		0x1089	ERR_PARSE			Error In The Definition Files.
MOSCAD		0x108A	ERR_SITE			Non Existing Site ID.
MOSCAD		0x108B	ERR_SYSTEM			The API Internal Data Structure Is Corrupted
MOSCAD		0x108C	ERR_ROW			Invaid Row
MOSCAD		0x108D	ERR_COLUMN			Invalid Column
MOSCAD		0x108E	ERR_TABLE			Table Number Error - Out Of Range Or Table Is Not A Valid Communication Table
MOSCAD		0x108F	ERR_COORD			Data Element Coordinate Error - Site ID Or Table Number Unknown.
MOSCAD		0x1090	ERR_ORDER			Not Currently Used (ORDER)

PROTOCOL	MASK	ERROR	MESSAGE	REFERENCE	ACTION	COMMENT
MOSCAD		0x1091	ERR_PARTIAL_RECV			Received Only Part Of The Requested Length
MOSCAD		0x1092	ERR_WARNING			There Are Warning Indications.
MOSCAD		0x1093	ERR_MESSAGE			Error In The Message Structure. The Data Entities Inside The API Message Buffer Should Be Of Complete RTU Rows, Columns Or Tables (Or A Single One).
MOSCAD		0x1094	ERR_FAIL_MESSAGE			The Current MDLC Block Returned An Error Status - Could Not Parse The Block.
MOSCAD		0x1095	ERR_LAST_ENTITY			Not Currently Used (LAST ENTITY)
MOSCAD		0x1096	ERR_IP			Illegal IP Address Or IP Address Format
MOSCAD		0x1097	ERR_USED			Not Currently Used (USED)
MOSCAD		0x1098	ERR_CHANNEL			Error Returned From A BSD Socket Library Routine
MOSCAD		0x1099	ERR_MISMATCH			API Internal MOSCAD Field System Description Does Not Match The MOSCAD Field.
MOSCAD		0x109A	ERR_ELEMENT			Data Entity Element Type Er
MOSCAD		0x109B	ERR_HASH_FULL			The Hash Table Is Full
MOSCAD		0x109C	ERR_PARAM			Parameter Error.
MOSCAD		0x109D	ERR_NAME_SIZE			Name Is Too Int32.
MOSCAD		0x109E	ERR_ENTRY			The Entry Requested Was Not Found

3.14 Debug Messages

None

3.15 Stats Special Counters

Number	Label	Purpose/Meaning of this counter
0	TOTAL ROWS IN CACHE	Total number of allocated table row structures in the cache
1	NUM ROWS CURRENTLY ACTIVE	The number of active rows found while executing the polling algorithm (value accumulates during the cycle, and is then overwritten by the next cycle)
2	NUM OUTSTANDING REQUESTS	Number of outstanding physical requests to a gateway
3	DATA Rx THREAD WATCHDOG	Watchdog counter used to indicate whether the incoming data reception thread is operational
4	CACHE UPDATE THREAD W.DOG	Watchdog counter used to indicate whether the cache freshness check thread is operational
5	CHAN. INIT THREAD ACTIVE	Flag to indicate whether the channel initialisation thread and socket creation function is running (should only be true when a channel is being initialised)
6	CHAN. INIT CNTR	Counter to indicate how many times the channel initialisation function has been called
7	INIT STATE (1-16)	Counter to indicate which of the 16 states (stages) the initialisation function is operating at (should be at 16 during normal operation after initialisation).
8	DATA BUFFER ACQUIRED CNTR	Counter to indicate how many times buffer transfer has been undertaken between the gateway and the driver.
9	UNSOLICITED RESPONSES RxD	Counter which increments whenever an unsolicted response is received
10	TOTAL UNIT POLLS	Counter which increments on every poll to a unit
11	LAST API ERROR	Holds the most recent API error
12	DRIVER EVENT QUE LENGTH	This value applies to entire driver, not just to this channel. The number of events held in the driver's event queue.
13	DRIVER DELAYED EVENT QUE	This value applies to entire driver, not just to this channel. The number of events currently being delayed prior to being placed on the driver's event queue (for repeating alarms)
14	DRIVER TOTAL EVENTS	This value applies to entire driver, not just to this channel. The total number of events the driver has processed.

3.16 Hints and Tips

The use of the MOSCAD system with this driver provides a lot of flexibilty, but requires a knowledge of the MOSCAD system. It is assumed that the user of this driver is familiar with MOSCAD systems and the configuration of MOSCAD RTU's. It is strongly recommended that the user reviews all of the information provided here before developing a project with this driver.

If the Syslog is reporting "Error: Driver is not responding", then the driver may be unable to establish a TCP/IP connection. This is due to the driver being held up by the TCP/IP timeout, thus preventing the driver from responding back to Citect within its own timeout, causing Citect to issue an error.

Ensure that IP address has been set correctly in Citect and Gateway, and that all equipment is connected correctly.

Ensure that the configuration file <MDLC_Pra>.cfg have been set correctly as explained in Section 5.10, and that they have been included in the working Bin directory of Citect. The driver is dependant on these files being set correctly in order for it to correctly communicate with the RTU. Incorrect settings on these files will generate errors in the Syslog.

The API states that a user table configured in the RTU for use by Citect, should not have a discrete column as the first and last column, except if the table contains only discrete columns or only one column.

The driver will support the built in gateway redundancy provided by the MDLC API. From the users point of view this means that a backup gateway can be configured in the ports form, for the primary gateway, as a special option. A backup gateway should not be entered as a separate ports form entry.

There is a maximum of 250 RTUs supported per gateway.

☞ Radio transmission rates are typically very slow. In addition, RTUs are capable of independently "bursting" unsolicited data back to the gateway. If the system is not setup correctly, comms traffic may be saturated to the point that collisions reduce effective throughput to a trickle. This will also generate numerous errors and other complications. It must be verified that the timing of the comms system has been setup adequately before trying to faultfind any driver behaviour.

☞ When using redundancy across two IO Servers, the standby units will show their status on the devices form as "standby" when they are available (regardless of whether they are actually in standby mode or active). Use the "client using" field to determine if they are active.

3.16.1.1.1 Setup Hints

- The critical RTU must be the first device in the IO Devices form.
- A single critical RTU must be defined in the cfg setup file for the primary (ie all but one set to "noncritical"). The cfg setup file for the secondary must not contain any critical RTUs (ie all set to "noncritical").
- The IO Server holding the primary devices should have a "*GatewayMode*" INI parameter set to "1" in the INI section representing the first port. The IO Server holding the standby devices must have a "GatewayMode" INI parameter set to "2" in the INI section representing the first port. For example, if the INI files define the "*Station*" as "[MOSCAD] Station=Gamma", then the primary should have an INI section labelled [Gamma0] with a parameter "GatewayMode=1". Similarly, the secondary should also have an INI section labelled [Gamma0] but with a parameter "GatewayMode=2".
- Both IO Servers must have a " Multiplexed " INI parameter set to "0" in the INI section representing the first port. This indicates that both channels are sharing the physical communication link.
- The driver must be configured to provide adequate radio idle time (ie padding between polls). If the RTUs have not been configured to "burst" data back to the gateway, then only the standby needs to separate the polls to allow for the primary to perform its health check on the critical RTU. If "bursting" is used, then polling settings for both the primary and standby must allow enough radio silence for this to occur.

- Each gateway to be configured for “standalone” mode and each to have unique SiteIds.
- Each gateway must have its “health check” flag disabled.
- Each RTU has to be configured to detect a broadcast from the driver and to redirect its “burst” target to the relevant SiteId. This is required as only one gateway is active at a time, and the RTUs must burst their data back to the correct gateway. This is only a minor amount of RTU configuration. Refer to “Overview of redundancy using two standalone gateways”.
- The Citect project will require cicode that broadcasts instructions to the RTU, causing them to redirect to a new burst target. This is required for failover. It must also periodically broadcast to cover the case of an RTU that may have been offline at change over. Tom Sabitzer, of the Lieden office, has written Cicode that provides this functionality. It has been tested exhaustively during the FAT, and is part of the solution for the Getronics project. Contact Tom for a copy of this cicode (royalties may apply!!).
- Any cicode commands issuing tag reads or tag writes should check the status virtual tag to ensure the device is available before actually issuing a command.
- It is recommended that the driver be set with 0 retries due to the reasons described in “Issues”.
- The “Timeout” parameter must be set large enough to cater for the indeterminate nature of the gateway (as described in “Issues”). If required use the additional “TimeoutExt” parameter.
- The “Watchtime” parameter, for the IO Server holding the primary devices, must be set large enough so that the regular healthcheck of the critical RTU (when the channel is offline) will not interfere with the polling operation of the standby. A typical example may be 180 seconds.
- If writing application cicode that will issue commands resulting in physical communication to devices, ensure that the command rate is throttled to prevent swamping the diver with requests. The polling regime will also have to be tuned to free up enough idle time if this will occur on a permanent basis. Due to the slow nature of the physical comms, providing too high a vale of ““MaxPending” will run the risk of the driver buffering up so many requests that they can not all be processed for the IO Server in time. This will result in the IO Server deducing that the driver has ceased functioning and it will be reset. However, not having a high enough value will result in a bottleneck as the IO Server will have all of its requests tied up waiting for physical comms, with no requests spare to read values from cache. This will cause very strange behaviour that is peculiar to RTU comms (first observed in Perth on the Hunter WaterTech DNP project). The page update will freeze and the client may actually #COM its values even though the driver and IO Server are not reporting any errors!!!